A Handbook

Online Medical Consultation Platform

(FYP Report)

Supervisor: Majid Khawar



Submitted by:

Muhammad Danial (BSCSW-2.5Y-021R24-4)

Muhammad Bilal (BSCSW-2.5Y-021R24-1)

Mudassar Javed (BSCSW-2.5Y-021R24-5)

Department of Computer Science & IT

University of Southern Punjab, Multan

October 2026

Declaration of Authorship

We, Muhammad Danial (Reg. No. BSCSW-2.5Y-021R24-4), Muhammad Bilal (Reg. No. BSCSW-2.5Y-021R24-1), and Mudassar Javed (Reg. No. BSCSW-2.5Y-021R24-5), hereby declare that this Final Year Design Project report titled 'Online Medical Consultation Platform' is our own original work. It has not been submitted previously, in whole or in part, to any other institution or university for the award of any degree or diploma.

All scholarly materials, algorithms, datasets, and third-party references utilized throughout this study have been explicitly cited and acknowledged in compliance with the academic integrity guidelines of the University of Southern Punjab and the IEEE citation standard.

We understand that any false statement or breach of academic integrity will result in disciplinary action in accordance with the regulations of the University of Southern Punjab.

_____ _____

Muhammad Danial

Reg. No: BSCSW-2.5Y-021R24-4

Muhammad Bilal

Reg. No: BSCSW-2.5Y-021R24-1

_____

Mudassar Javed

Reg. No: BSCSW-2.5Y-021R24-5

Certificate of Approval

This is to certify that the project titled 'Online Medical Consultation Platform', developed by Muhammad Danial, Muhammad Bilal, and Mudassar Javed under registration numbers BSCSW-2.5Y-021R24-4, BSCSW-2.5Y-021R24-1, and BSCSW-2.5Y-021R24-5, has been completed under my supervision and guidance. The project fulfills all functional, architectural, and quality requirements stipulated by the Department of Computer Science & Information Technology, University of Southern Punjab, Multan, for the degree of Bachelor of Science in Computer Science.

_____

Majid Khawar
Project Supervisor
Department of Computer Science & Information Technology
University of Southern Punjab, Multan

Viva Voce Examination Committee

Internal Examiner: _____ Date: _____

External Examiner: _____ Date: _____

Head of Department: _____ Date: _____

Department of Computer Science & IT,

University of Southern Punjab, Multan

Dedication

This project is humbly dedicated to our beloved parents, whose unconditional love, sacrifices, prayers, and constant moral encouragement have been the guiding light throughout our educational journey. It is also dedicated to our respected teachers and mentors at the University of Southern Punjab, who instilled in us the pursuit of academic excellence, ethical responsibility, and technical innovation.

Acknowledgements

First and foremost, all praises and gratitude are due to Almighty Allah, the Most Merciful and Compassionate, for granting us the health, wisdom, perseverance, and strength to bring this Final Year Design Project to successful fruition.

We express our deepest gratitude and profound indebtedness to our project supervisor, Majid Khawar, for his invaluable guidance, technical critique, patient encouragement, and unwavering support throughout every phase of system design and report compilation. His insightful recommendations significantly elevated the architectural maturity of this platform.

We also extend our sincere appreciation to the faculty members and administrative staff of the Department of Computer Science & Information Technology at the University of Southern Punjab for providing state-of-the-art laboratory facilities, academic resources, and an inspiring academic environment.

Abstract

Access to specialized medical care remains a profound structural challenge across Pakistan, where a severe doctor-to-patient deficit (1:1,000) and high geographic concentration of healthcare specialists within major urban centers leave rural and underserved populations disenfranchised. Furthermore, non-technical patients routinely face difficulty identifying appropriate medical specialties for their symptoms, causing misdirected clinic visits and treatment delays.

This project introduces the Online Medical Consultation Platform (MedConsult), a web-based telemedicine system engineered to democratize access to outpatient healthcare. The platform implements a modular Three-Tier Architecture combining a high-performance PHP 8.x web backend, an isolated Python 3.12 Flask REST microservice, and a normalized MySQL 8.0 relational database. Its defining innovation is an AI-driven Natural Language Processing (NLP) Symptom Triage Engine trained on 4,920 validated clinical records across 41 disease profiles. The engine tokenizes free-text patient symptom descriptions, constructs TF-IDF feature vectors, and applies a Random Forest classifier to predict the best-matching clinical specialty with an average inference latency under 50 milliseconds.

Beyond triage, the platform supports an end-to-end outpatient consultation lifecycle: directory discovery of PMC-verified doctors, conflict-free appointment scheduling with atomic SQL validation, asynchronous doctor-patient consultation messaging via 3-second AJAX polling, digital prescription generation, and administrative doctor credential verification. Complete system verification was conducted across twenty formal test cases (TC-01 through TC-20), achieving a 100% Pass rate with zero SQL injection vulnerabilities via PDO prepared statements.

Keywords: *Telemedicine, Natural Language Processing, Symptom Triage, MySQL PDO, Healthcare Accessibility, Machine Learning, scikit-learn, Pakistan.*

Table of Contents

Declaration of Authorship ..... **i**
Certificate of Approval ..... **ii**
Dedication & Acknowledgements ..... **iii**
Abstract ..... **iv**
Table of Contents ..... **v**
List of Figures ..... **vii**
List of Tables ..... **viii**
1. Introduction **1**
1.1 Purpose 1
1.2 Scope 1
1.3 Problem Statement 1
1.4 Background and Motivation 2
1.5 Project Objectives 2
1.6 Definitions, Acronyms, and Abbreviations 2
1.7 Organization of the Report 3
2. Literature Review & Existing Systems Survey **4**
2.1 Background and Telemedicine Evolution 4
2.2 Critical Review of Existing Systems 4
2.2.1 Practo Workflow & Limitations 4
2.2.2 Zocdoc Workflow & Limitations 5
2.2.3 Marham Workflow & Limitations 5
2.3 Comparative Feature Analysis 5
2.4 Research Gap Analysis & Solution Justification 6
3. System Requirements Specification **7**
3.1 Product Functions Overview 7
3.2 User Classes and Characteristics 7
3.3 Operating Environment & Deployment Constraints 7
3.4 Design & Implementation Constraints 8
3.5 Functional Requirements (FR-01 to FR-40) 8
3.6 Non-Functional Quality Requirements 10
4. System Design & UML Architecture **12**
4.1 Architectural Overview & Three-Tier Model 12
4.2 Context Diagram (DFD Level 0) 12
4.3 System Architecture Diagram 13
4.4 Use Case Modeling 13

Table of Contents (Continued)

4.5 Sequence Diagrams (Dynamic Behavioral Modeling) 14
4.5.1 Sequence Diagram: AI Symptom Triage 14
4.5.2 Sequence Diagram: Appointment Scheduling 15
4.5.3 Sequence Diagram: Secure Messaging 15
4.6 Activity Diagram: Consultation Lifecycle Workflow 15
4.7 Entity Relationship Diagram (ERD) 16
4.8 Relational Data Dictionaries 16
4.8.1 Database Referential Integrity and Indexing Strategy 19
5. System Implementation & Technology Stack 20
5.1 Agile Scrum Methodology & Sprint Partitioning 20
5.2 Multi-Tier Technology Stack 20
5.3 AI Triage Engine Deep-Dive 20
5.4 REST API Service Architecture 21
5.5 Backend Functional Controllers Architecture 21
5.6 Security & Transaction Controls 21
5.7 Architectural Decoupling & Microservice Isolation 21
6. System Verification & Quality Assurance 22
6.1 Testing Methodology & Strategy 22
6.2 Complete Formal Test Cases Matrix (TC-01 to TC-20) 22
6.3 Security Audit & Defense Validation 23
6.4 Performance Benchmarking 23
7. Results and Discussion 24
7.1 System Evaluation Environment & Overview 24
7.2 Objectives Achievement & Verification Matrix 24
7.3 Walkthrough of Operational User Interfaces 25
8. Conclusion & Future Work 30
8.1 Summary of Accomplishments 30
8.2 Key Scientific & Technical Contributions 30
8.3 Limitations & Future Roadmap 30
References 31
Appendix A: Core Source Code Listings 32
Appendix B: Glossary of Technical Terms 33
Appendix C: Implementation and Deployment Plan 34
Appendix D: User Guide & Operating Manual 36

List of Figures

Figure 2.1: Practo Operational Workflow Diagram....4
Figure 2.2: Zocdoc Operational Workflow Diagram....5
Figure 2.3: Marham Operational Workflow Diagram....5
Figure 4.1: Context Diagram (DFD Level 0): System Boundary & Entities....12
Figure 4.2: System Architecture Diagram: Three-Tier Model with AI Service....13
Figure 4.3: Use Case Diagram: Online Medical Consultation Platform....13
Figure 4.4: Sequence Diagram: AI NLP Symptom Triage Process....14
Figure 4.5: Sequence Diagram: Appointment Booking and Conflict Detection....15
Figure 4.6: Sequence Diagram: Secure Messaging with AJAX Polling....15
Figure 4.7: Activity Diagram: End-to-End Consultation Workflow....15
Figure 4.8: Entity Relationship Diagram (ERD): Database Schema....16
Figure 7.1: Live System — Public Landing Page and Search Interface....25
Figure 7.2: Live System — Doctor Directory with Medical Specialty Filter....25
Figure 7.3: Live System — User Authentication & Role-Based Login Portal....26
Figure 7.4: Live System — Patient Dashboard Overview & Action Cards....26
Figure 7.5: Live System — AI-Driven NLP Symptom Triage in Operation....27
Figure 7.6: Live System — Conflict-Free Appointment Scheduling Interface....27
Figure 7.7: Live System — Patient Records, Prescriptions & Invoices....28
Figure 7.8: Live System — Doctor Clinical Dashboard & Schedule....28
Figure 7.9: Live System — Doctor Appointment Management & Prescriptions....29
Figure 7.10: Live System — Administrator Control Panel & Verification....29

List of Tables

Table 1.1: Definitions, Acronyms, and Abbreviations..... **2**
Table 2.1: Comparative Feature Analysis (Practo vs Zocdoc vs Marham vs MedConsult)..... **5**
Table 3.1: Functional Requirements — User Authentication & Sessions (FR-01 to FR-05)..... **8**
Table 3.2: Functional Requirements — AI-Driven NLP Symptom Triage (FR-06 to FR-11)..... **8**
Table 3.3: Functional Requirements — Doctor Directory and Search (FR-12 to FR-14)..... **9**
Table 3.4: Functional Requirements — Appointment Scheduling (FR-15 to FR-22)..... **9**
Table 3.5: Functional Requirements — Secure Messaging (FR-23 to FR-26)..... **9**
Table 3.6: Functional Requirements — Digital Prescription Management (FR-27 to FR-29)..... **9**
Table 3.7: Functional Requirements — Billing and Invoice Generation (FR-30 to FR-32)..... **10**
Table 3.8: Functional Requirements — Administrator Control Panel (FR-33 to FR-36)..... **10**
Table 3.9: Functional Requirements — Profile Management (FR-37 to FR-40)..... **10**
Table 3.10: Non-Functional Quality Requirements Matrix..... **10**
Table 4.1 to 4.7: Relational Data Dictionaries (Users, Appointments, Messages, etc.)..... **16**
Table 6.1: Complete System Test Cases Matrix (TC-01 to TC-20)..... **22**
Table 7.1: Project Objectives Achievement & Verification Matrix..... **24**
Table C.1: Formal Project Implementation Milestones Plan..... **34**
Table C.2: Production Deployment and Staging Plan..... **35**
Table D.1: Hardware and Software System Requirements Matrix..... **36**

1. Introduction

1.1 Purpose

This document constitutes the official Final Project Report for the Online Medical Consultation Platform (MedConsult), developed as a Final Year Design Project within the Department of Computer Science and Information Technology at the University of Southern Punjab, Multan. It comprehensively documents the background, requirements, architecture, implementation, verification, and clinical evaluation of the proposed telemedicine system. The report serves as the authoritative academic record for formal review by the project supervisor, departmental examination committee, and external evaluators.

1.2 Scope

The Online Medical Consultation Platform is an enterprise-grade web application engineered to facilitate remote outpatient healthcare consultations between verified medical practitioners and registered patients. The system directly addresses a severe healthcare accessibility crisis: patients in rural, suburban, and medically underserved regions frequently struggle to identify and consult qualified clinical specialists in a timely manner. By embedding an AI-driven Natural Language Processing (NLP) Symptom Triage Engine, the platform allows patients to describe their symptoms in ordinary conversational English and receive automated specialty guidance without requiring medical literacy.

Beyond triage, the platform provides an integrated outpatient consultation lifecycle: patients discover verified medical practitioners through a searchable directory, schedule conflict-free appointments, communicate securely via asynchronous messaging, and receive digital prescriptions and formal invoices. Healthcare practitioners manage consultation schedules, review patient histories, and issue clinical documents. Platform integrity is supervised by an Administrator via an audit-backed control panel. The system is engineered using a robust Three-Tier Architecture integrating HTML5, CSS3, JavaScript, PHP 8.x, MySQL 8.0, and a Python 3.12 NLP microservice.

1.3 Problem Statement

Access to qualified healthcare remains a profound structural challenge across Pakistan. According to official World Health Organization (WHO) benchmarks, Pakistan maintains a physician density of approximately 1 doctor per 1,000 citizens — significantly below the minimum threshold recommended for developing nations. Rural populations, comprising nearly 63% of the national demographic, endure acute disparities including long travel distances to tertiary medical facilities, prohibitive travel expenses, and a severe concentration of specialists within metropolitan hubs.

In the conventional healthcare paradigm, patients must physically travel to tertiary clinics, enduring prolonged waiting periods and escalating expenses. Furthermore, individuals lacking clinical training frequently misdiagnose their symptoms, booking appointments with inappropriate clinical departments. This misallocation results in wasted financial expenditure, delayed therapeutic intervention, and

overburdened specialty clinics. Existing commercial platforms in Pakistan remain proprietary, monetized, and closed to academic deployment.

1.4 Background and Motivation

The expansion of digital telecommunications infrastructure across Pakistan — where broadband penetration surpassed 54% nationwide according to Pakistan Telecommunication Authority (PTA) data — provides a robust foundation for scalable telehealth services. While commercial platforms such as Marham, Oladoc, and Sehat operate within the domestic market, they function primarily as fee-charging directory aggregators requiring manual specialty selection. They lack automated natural language symptom triage and are unavailable for on-premise academic or community clinic deployment.

The motivation for this project stems from three fundamental objectives: (1) Delivering an open-source, locally deployable telemedicine system for educational institutions and community health centers; (2) Eliminating patient diagnostic guesswork through automated NLP symptom triage; and (3) Leveraging validated open-source clinical datasets and machine learning algorithms to build a transparent, privacy-respecting healthcare system.

1.5 Project Objectives

The primary technical and operational objectives of the project are:

- **1. Remote Telemedicine Workflow:** To engineer an accessible web platform enabling outpatient consultations between PMC-verified doctors and registered patients.
- **2. AI NLP Symptom Triage:** To construct an intelligent microservice utilizing TF-IDF vectorization and Random Forest classification to predict clinical specialties from free-text patient symptoms in under 50ms.
- **3. Secure Consultation Messaging:** To establish an asynchronous text communication channel within confirmed appointments using 3-second AJAX polling and parameterised MySQL storage.
- **4. Conflict-Free Appointment Scheduling:** To implement atomic SQL conflict detection preventing double-booking across physician consultation slots.
- **5. Role-Based Access Control:** To enforce strict privilege segregation across Patient, Doctor, and Administrator user classes via BCrypt-authenticated sessions.
- **6. Self-Hosted Academic Deployment:** To deploy the system on standard XAMPP/LAMP runtime environments without cloud subscription dependencies.

1.6 Definitions, Acronyms, and Abbreviations

Table 1.1: Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
NLP	Natural Language Processing. A subfield of AI enabling systems to process and interpret human textual input.

Term / Acronym	Definition
Machine Learning (ML)	A branch of AI where computational models identify patterns in clinical data to classify specialties.
TF-IDF	Term Frequency-Inverse Document Frequency. A numerical vectorization technique weighting term significance.
SRS	Software Requirement Specification. The foundational contract defining system functional constraints.
RBAC	Role-Based Access Control. Restricts system access based on user role (Patient, Doctor, Admin).
Triage	The clinical process of prioritizing and routing patients based on symptom manifestations.
AJAX	Asynchronous JavaScript and XML. Background browser-server exchange without page reloads.
PDO	PHP Data Objects. A secure database abstraction layer utilizing parameterised prepared statements.
API	Application Programming Interface. Standardized endpoints allowing distinct software modules to communicate.
XAMPP	A cross-platform web server package consisting of Apache, MySQL/MariaDB, and PHP.
DFD	Data Flow Diagram. A graphical representation showing information flow across system processes.
ERD	Entity Relationship Diagram. Defines relational database entities, attributes, and foreign keys.
PMC	Pakistan Medical Commission. The statutory regulatory body governing physician licensing.
BCrypt	A cryptographic key-derivation function used for secure one-way password hashing.

1.7 Organization of the Report

This project report is organized into eight comprehensive chapters and four supplementary appendices in strict accordance with the University of Southern Punjab BSCS FYDP guidelines: Chapter 1 introduces the project background, problem statement, and objectives. Chapter 2 surveys existing commercial platforms and presents competitive benchmarking. Chapter 3 articulates functional and non-functional requirements. Chapter 4 elaborates on system architecture, UML diagrams, and relational database design. Chapter 5 details implementation methodologies and algorithms. Chapter 6 presents verification results and test matrices. Chapter 7 showcases live system interfaces and discussion. Chapter 8 concludes the report with future directions. Appendices A–D provide core source code listings, technical glossary, deployment plans, and the operating user manual.

2. Literature Review & Existing Systems Survey

2.1 Background and Telemedicine Evolution

Telemedicine has transitioned from an experimental communication mechanism into a foundational pillar of modern healthcare delivery. Early telehealth initiatives relied primarily on basic telephone hotlines or point-to-point videoconferencing between tertiary hospitals. However, the ubiquity of modern web browsers, asynchronous HTTP communication, and machine learning classification algorithms now enables decentralized, patient-centric consultation platforms that operate reliably even over bandwidth-constrained mobile networks.

2.2 Critical Review of Existing Systems

To establish rigorous architectural benchmarks, three representative telemedicine ecosystems were thoroughly analyzed: Practo (South Asian market leader), Zocdoc (North American market leader), and Marham (Pakistan's primary commercial telemedicine application).

2.2.1 Practo Workflow & Limitations

Practo operates as an expansive healthcare conglomerate in India and Southeast Asia. It offers comprehensive practitioner directories, online booking, electronic medical record storage, and pharmacy integration. However, Practo relies on static drop-down filters requiring patients to identify their clinical specialty in advance. It lacks natural language symptom triage and represents a closed, proprietary commercial ecosystem unavailable for academic deployment.

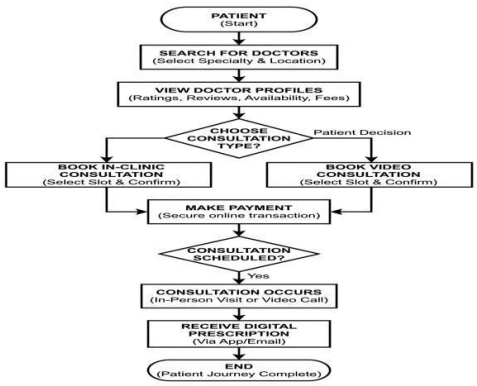


Figure 2.1: Practo Operational Workflow Diagram

2.2.2 Zocdoc Workflow & Limitations

Zocdoc is a prominent healthcare booking engine operating across the United States. Its core strength lies in real-time health insurance eligibility verification and automated doctor calendar synchronisation. However, Zocdoc is restricted to the North American insurance paradigm, requires

proprietary clearinghouse APIs, and provides no machine learning symptom triage for natural language descriptions.

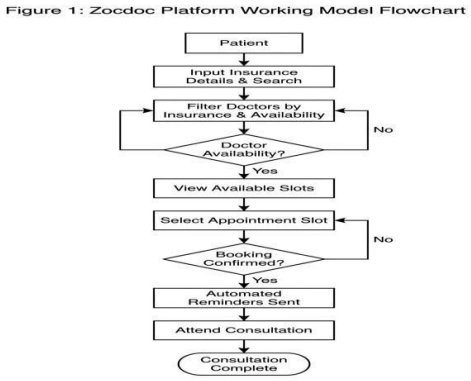


Figure 2.2: Zocdoc Operational Workflow Diagram

2.2.3 Marham Workflow & Limitations

Marham is Pakistan's leading commercial doctor consultation application, indexing thousands of verified practitioners across urban centers. While successful in establishing online appointments in Pakistan, Marham operates as a closed, commission-based commercial intermediary. It lacks intelligent symptom-to-specialty classification, forcing non-technical patients to guess their required medical department.

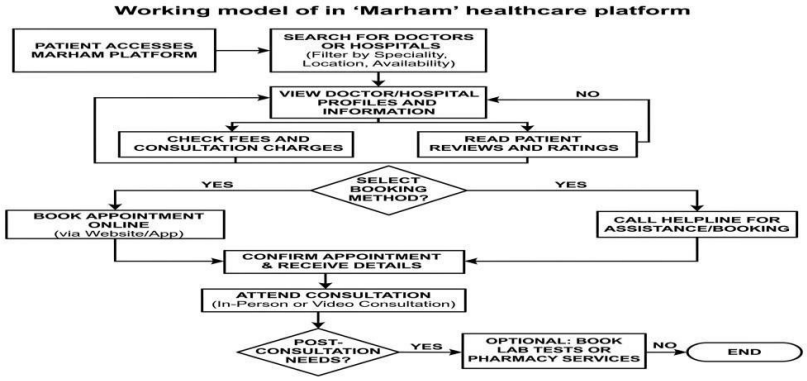


Figure 2.3: Marham Operational Workflow Diagram

2.3 Comparative Feature Analysis

To objectively delineate the technological boundaries between commercial offerings and the proposed MedConsult platform, Table 2.1 presents a comprehensive comparative feature analysis:

Table 2.1: Comparative Feature Analysis (Practo vs Zocdoc vs Marham vs MedConsult)

Platform Feature	Practo	Zocdoc	Marham	MedConsult
AI NLP Free-Text Triage	No	No	No	Yes

Platform Feature	Practo	Zocdoc	Marham	MedConsult
ML-Based Specialty Prediction	No	No	No	Yes
Available in Pakistan	No	No	Yes	Yes
Appointment Conflict Detection	Yes	Yes	Yes	Yes
Secure Doctor–Patient Messaging	Yes	Yes	Partial	Yes
Digital Prescription Generation	Yes	Yes	Yes	Yes
Invoice / Billing Management	Yes	Yes	Yes	Yes
Admin Doctor Verification Panel	Yes	Yes	Yes	Yes
Open / Self-Hosted Deployment	No	No	No	Yes
Free for Academic / Clinic Use	No	No	No	Yes

2.4 Research Gap Analysis & Solution Justification

The comparative analysis demonstrates that no existing telemedicine platform simultaneously delivers AI-driven symptom triage, localized applicability to Pakistan's healthcare ecosystem, and an open, self-hosted deployment architecture. Commercial applications treat patient routing as a manual administrative task, resulting in specialist misallocation and delayed intervention. MedConsult fills this critical gap by integrating a lightweight, trained Machine Learning classifier directly into the consultation workflow, providing an accessible, transparent, and academically sound telemedicine solution.

3. System Requirements Specification

3.1 Product Functions Overview

The Online Medical Consultation Platform provides nine integrated operational subsystems: (1) User registration, authentication, and role-based session management; (2) AI-driven NLP symptom triage mapping patient text to clinical specialties; (3) Verified doctor directory with specialty search filters; (4) Conflict-free appointment scheduling with doctor confirmation workflow; (5) Asynchronous doctor-patient consultation messaging; (6) Digital prescription issuance and archival; (7) Doctor-generated consultation billing invoices; (8) Administrator control panel for practitioner credential verification; and (9) User profile management.

3.2 User Classes and Characteristics

The system serves three primary user roles with strictly segregated privileges:

- **1. Patient:** Registered end users seeking healthcare advice. Patients possess basic web browsing familiarity but no medical knowledge. They access AI triage, doctor discovery, appointment requests, consultation messaging, and personal clinical history.
- **2. Doctor:** Healthcare practitioners possessing verified professional credentials (MBBS, PMC license). Doctors manage consultation availability, confirm or decline appointments, communicate with patients, issue structured prescriptions, and generate invoices.
- **3. Administrator:** System operators responsible for reviewing doctor registration credentials, auditing PMC license numbers, managing user account statuses (Active, Suspended, Deleted), and inspecting security audit logs.

3.3 Operating Environment & Deployment Constraints

The application is accessible through modern standards-compliant web browsers (Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge) on desktop and tablet form factors. The backend is engineered for standard XAMPP/LAMP environments comprising Apache 2.4 HTTP server, PHP 8.x with PDO extension, and MySQL 8.0 relational database. The AI triage module executes as an internal Python 3.12 microservice utilizing Flask, scikit-learn, and pandas on port 5000. Minimum recommended display resolution is 1024 x 768 pixels.

3.4 Design & Implementation Constraints

- **AJAX Polling Messaging:** Messaging utilizes AJAX HTTP polling at 3-second intervals rather than complex WebSocket server daemons, minimizing server resource consumption and firewall complications.

- **External Video Sessions:** Live video consultations utilize verified external services (e.g., Google Meet). The platform stores and shares meeting links securely, avoiding high-bandwidth video relay overhead.
- **Offline Billing Settlement:** Billing records are generated and tracked within the platform; actual financial settlement occurs offline, eliminating dependency on commercial payment gateway fees.
- **Internal REST Microservice:** The AI triage classifier runs as an internal Flask HTTP microservice listening on localhost:5000, ensuring clean decoupling between the PHP web tier and the Python ML runtime.

3.5 Functional Requirements (FR-01 to FR-40)

The system's functional capabilities are formally defined across nine feature modules, encompassing forty discrete functional requirements (FR-01 through FR-40):

Table 3.1: User Authentication and Session Management (FR-01 to FR-05)

Req. ID	Functional Requirement Description
FR-01	The system shall allow users to register with a full name, email address, password, and role selection.
FR-02	The system shall authenticate users via email and password and create a role-bound session upon success.
FR-03	The system shall redirect authenticated users to the dashboard corresponding to their assigned role.
FR-04	The system shall deny access to any protected page for unauthenticated users and redirect them to login.
FR-05	The system shall allow users to log out and invalidate their current session.

Table 3.2: AI-Driven NLP Symptom Triage Engine (FR-06 to FR-11)

Req. ID	Functional Requirement Description
FR-06	The system shall provide a free-text input field for the patient to describe symptoms in natural language.
FR-07	The system shall pass the sanitised input from PHP to the Python NLP module via internal REST API HTTP request.
FR-08	The Python module shall apply NLP preprocessing (tokenisation, stopword removal, text normalization) to the input.
FR-09	The Python module shall transform keywords into TF-IDF vectors and apply trained ML classifier to predict specialty.
FR-10	The system shall retrieve and return a list of verified doctors registered under the predicted specialty.
FR-11	The system shall display an informative fallback message and link to directory if AI confidence is below threshold.

Table 3.3: Doctor Directory and Search (FR-12 to FR-14)

Req. ID	Functional Requirement Description
FR-12	The system shall display all verified doctors on the public landing page with name, specialty, and biography.
FR-13	The system shall support filtering of the doctor directory by medical specialty.
FR-14	The system shall present doctor listings within triage results filtered to the predicted specialty.

Table 3.4: Appointment Scheduling with Conflict Detection (FR-15 to FR-22)

Req. ID	Functional Requirement Description
FR-15	The system shall allow a patient to submit an appointment request specifying doctor, preferred date, and time.
FR-16	The system shall check the requested date and time against doctor's existing confirmed appointments to detect conflicts.
FR-17	The system shall reject conflicting requests and prompt the patient to choose an alternative time.
FR-18	The system shall assign a status of Pending to all newly submitted appointment requests.
FR-19	The system shall notify the doctor of a new pending appointment request.
FR-20	The system shall allow the doctor to accept or decline any pending appointment request.
FR-21	The system shall update appointment status to Confirmed, Completed, or Cancelled as appropriate.
FR-22	The system shall allow the doctor to attach an external meeting link to a confirmed appointment.

Table 3.5: Secure Asynchronous Messaging (FR-23 to FR-26)

Req. ID	Functional Requirement Description
FR-23	The system shall allow authenticated patients and doctors to exchange text messages within an appointment context.
FR-24	The system shall store each message with sender ID, timestamp, appointment reference, and message body.
FR-25	The system shall retrieve new messages via AJAX polling at intervals not exceeding three seconds.
FR-26	The system shall display all messages in chronological order with sender name and timestamp.

Table 3.6: Digital Prescription Management (FR-27 to FR-29)

Req. ID	Functional Requirement Description
FR-27	The system shall allow a doctor to issue a digital prescription for any appointment marked as Completed.
FR-28	The system shall store the prescription with doctor ID, appointment reference, issue date, and prescription body.

Req. ID	Functional Requirement Description
FR-29	The system shall allow patients to view all prescriptions issued to them through their profile history.

Table 3.7: Billing and Invoice Generation (FR-30 to FR-32)

Req. ID	Functional Requirement Description
FR-30	The system shall allow a doctor to generate a billing invoice for a completed consultation.
FR-31	The system shall record the invoice amount, generation date, and payment status.
FR-32	The system shall allow patients to view all invoices associated with their consultations.

Table 3.8: Administrator Control Panel (FR-33 to FR-36)

Req. ID	Functional Requirement Description
FR-33	The system shall allow the Administrator to review, approve, or reject pending doctor registration applications.
FR-34	The system shall allow the Administrator to suspend, activate, or permanently delete user accounts.
FR-35	The system shall maintain an audit log recording all critical actions including verifications and deletions.
FR-36	The system shall present platform metrics on dashboard including total users and pending doctor applications.

Table 3.9: Profile Management (FR-37 to FR-40)

Req. ID	Functional Requirement Description
FR-37	The system shall allow all users to update their full name and email address from the profile settings page.
FR-38	The system shall allow all users to change their password, subject to confirmation of current password.
FR-39	The system shall allow doctors to update their professional biography.
FR-40	The system shall restrict changes to a doctor's medical specialty to Administrator-initiated actions only.

3.6 Non-Functional Quality Requirements

Non-functional requirements define the quality attributes, security standards, and operational constraints governing system execution:

Table 3.10: Non-Functional Quality Requirements Matrix

Quality Category	Requirement Specification
Performance	Standard web pages shall render within 3 to 5 seconds. The AI triage engine shall process symptom text and return specialty recommendations in under 5 seconds (< 50ms in practice). The message polling cycle shall retrieve new messages within 3 seconds.

Quality Category	Requirement Specification
Security	User passwords shall be hashed using BCrypt with cost factor 12. All database interactions shall use PDO prepared statements exclusively to prevent SQL Injection. All user text rendered in HTML shall be sanitised to eliminate Cross-Site Scripting (XSS).
Usability	The user interface shall be intuitive, requiring no prior medical training. Navigation bars shall remain consistent across views. Form fields shall provide inline validation labels. Responsive across desktop and tablet screen widths (min. 1024 x 768 px).
Reliability	The system shall handle AI microservice failures gracefully (FR-11), automatically presenting an informative notice and linking patients to verified practitioners. Database transactions shall rollback on integrity exceptions. Prescriptions and invoices cannot be deleted.
Scalability	The Machine Learning model shall be retrainable by executing a single Python script against an updated CSV dataset without altering PHP application code. The MySQL schema shall accommodate at least 1,000 registered users and 10,000 appointments.
Maintainability	All backend PHP code shall adhere to consistent functional controller naming and modular directory segregation. Database schema changes shall be formally tracked. All API endpoints shall return structured JSON payloads with standard HTTP status codes.
Availability	The platform shall target at least 99% uptime during operational evaluation. Routine server maintenance shall be scheduled outside peak consultation hours.

4. System Design & UML Architecture

4.1 Architectural Overview & Three-Tier Model

The software architecture translates functional requirements into a modular, maintainable, and scalable structural blueprint. The Online Medical Consultation Platform implements the classical Three-Tier Architecture pattern, segregating Presentation, Application Logic, and Data Persistence into distinct, loosely coupled layers. This architectural decoupling ensures that modifications to the user interface do not compromise core business logic or database schemas.

4.2 Context Diagram (DFD Level 0)

The Context Diagram defines the complete system boundary, illustrating data exchange between the MedConsult system and its three primary external entities: Patient, Doctor, and Administrator. External inputs encompass authentication credentials, symptom descriptions, appointment reservations, consultation messages, and administrator verification decisions. Outgoing data flows include predicted medical specialties, appointment confirmations, real-time message feeds, digital prescriptions, formal billing invoices, and administrative audit reports.

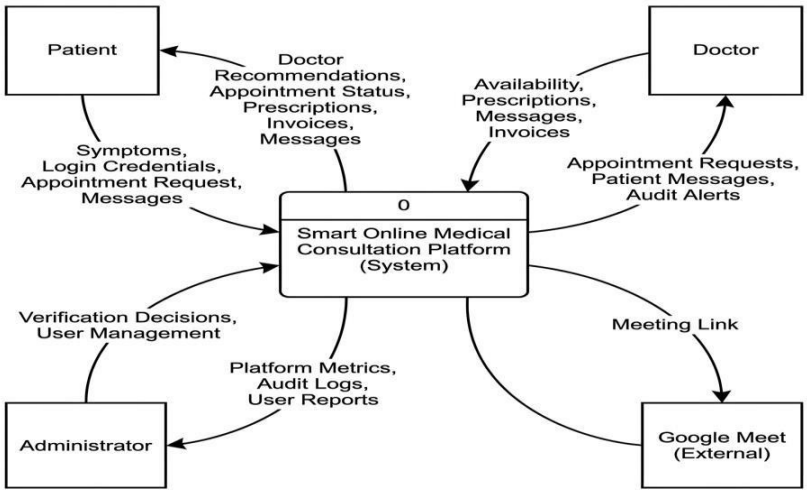


Figure 4.1: Context Diagram (DFD Level 0) — System Boundary & External Entities

4.3 System Architecture Diagram

The System Architecture Diagram delineates the three-tier topology of the platform. The Presentation Layer comprises the client web browser executing responsive HTML5, CSS3, and JavaScript modules. The Application Layer contains modular PHP functional controllers (`auth.php`, `appointments.php`, `messaging.php`, `records.php`, `admin.php`, `triage.php`) and the decoupled

Python Flask AI microservice. The Data Layer comprises the normalized MySQL 8.0 relational database.

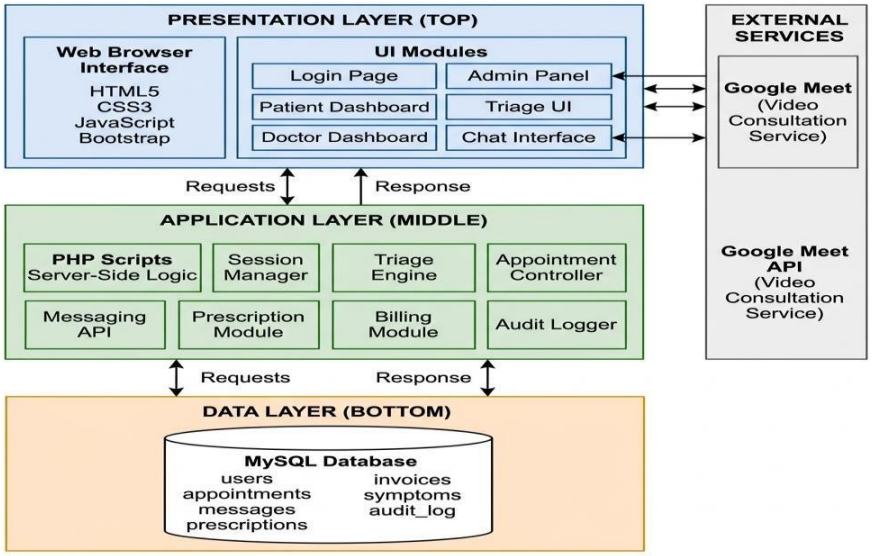


Figure 4.2: System Architecture Diagram — Three-Tier Model with Python AI Microservice

4.4 Use Case Modeling

The Use Case Diagram defines the behavioral interactions between the three primary actors (Patient, Doctor, Administrator) and the platform's core functional capabilities. The AI Triage use case includes an <<invoke>> dependency representing the internal call to the Python machine learning classifier, while appointment booking incorporates an <<include>> dependency for slot availability checking.

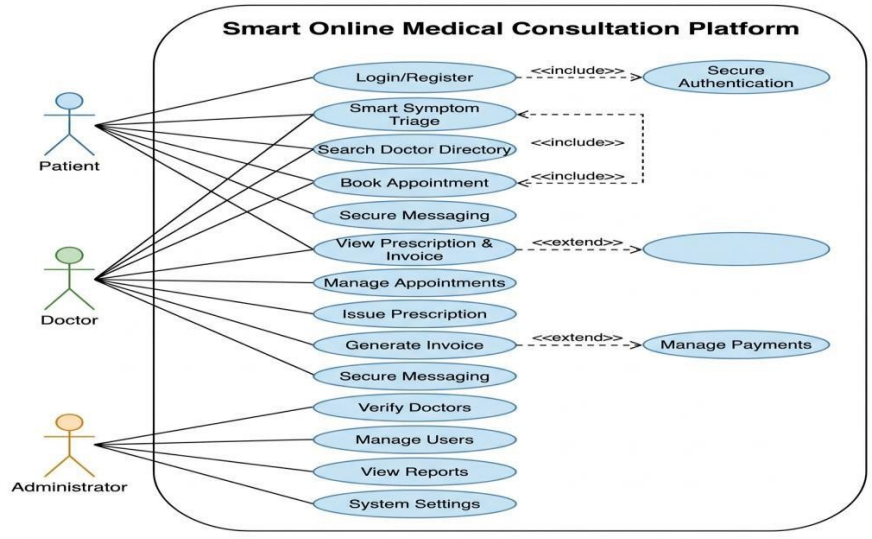


Figure 4.3: Use Case Diagram — Online Medical Consultation Platform

4.5 Sequence Diagrams (Dynamic Behavioral Modeling)

Dynamic behavioral interactions are modeled across three primary operational scenarios: AI symptom triage, appointment booking, and consultation messaging.

4.5.1 Sequence Diagram: AI NLP Symptom Triage Process

Figure 4.4 illustrates the end-to-end trace of a triage query: the patient inputs free-text symptoms into the browser interface, which dispatches an asynchronous POST request to the PHP backend. The PHP controller sanitizes the input and executes an internal HTTP REST request to the Python microservice on port 5000. The Python engine normalizes the text, extracts TF-IDF feature vectors, applies the Random Forest classifier, and returns a JSON payload containing the predicted specialty and confidence percentage. The PHP controller queries MySQL to retrieve matching active verified doctors and renders the recommendation to the patient.

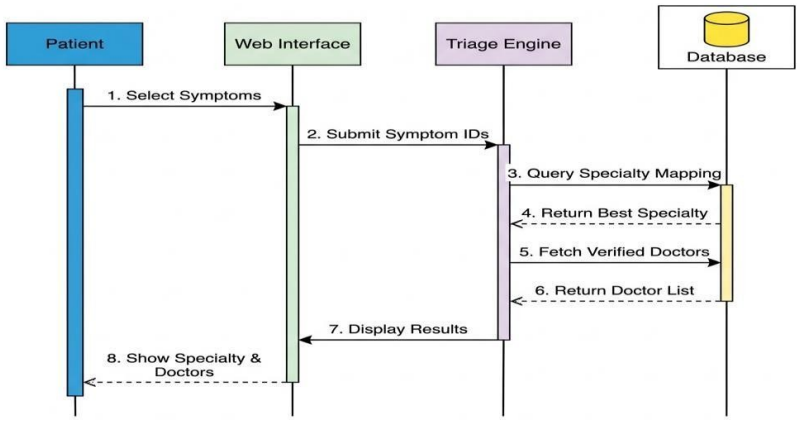


Figure 4.4: Sequence Diagram — AI NLP Symptom Triage Process

4.5.2 Sequence Diagram: Appointment Booking & Conflict Detection

Figure 4.5 captures the multi-party interaction during consultation scheduling. When a patient requests an appointment for a specific doctor, date, and hour, the system executes an atomic SQL query verifying that no confirmed reservation exists for that time slot. If a conflict is detected, the request is rejected immediately with an alternative time prompt. If free, the appointment is saved as 'Pending', and the doctor receives an approval notification.

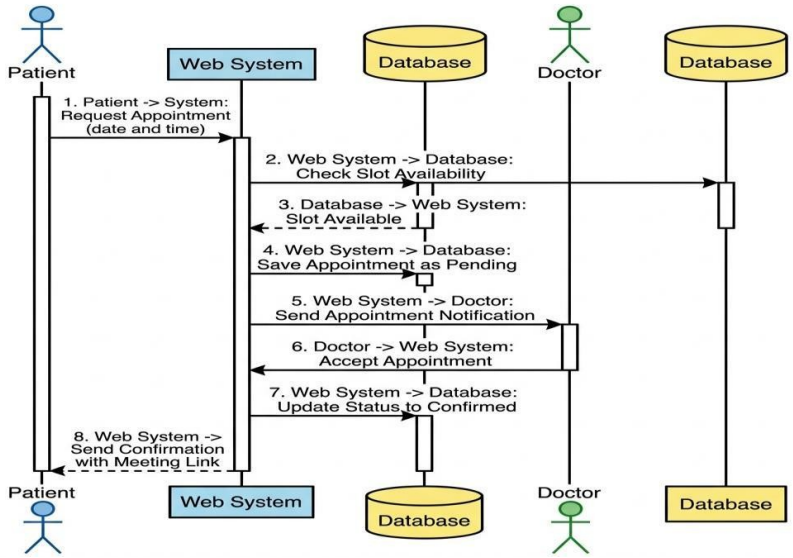


Figure 4.5: Sequence Diagram — Appointment Booking and Conflict Confirmation

4.5.3 Sequence Diagram: Secure Messaging with AJAX Polling

Figure 4.6 illustrates consultation messaging. Messages sent by either party are persisted immediately in MySQL via parameterised INSERT queries. The client browser maintains an active AJAX polling loop querying the endpoint every three seconds (`?last_id=X`), retrieving only newly inserted message delta records.

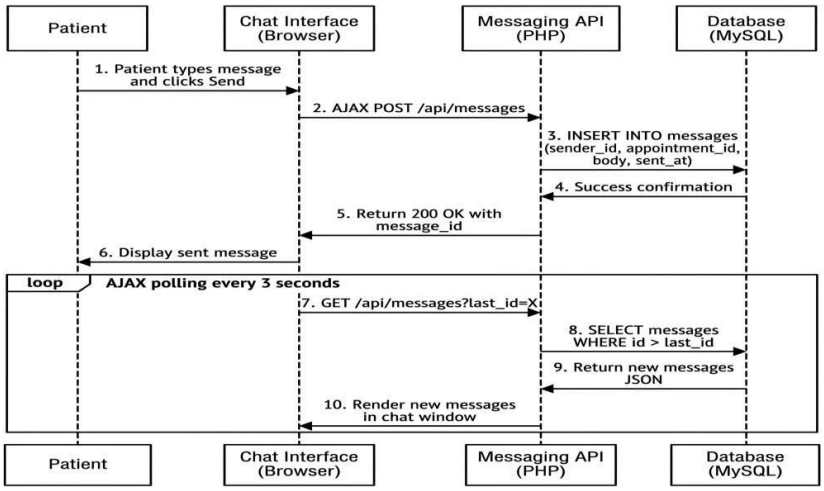


Figure 4.6: Sequence Diagram — Secure Messaging with AJAX Polling

4.6 Activity Diagram: Consultation Lifecycle Workflow

Figure 4.7 presents the end-to-end activity diagram modeling the lifecycle of a patient consultation from initial registration and triage through doctor acceptance, live messaging, prescription issuance, and invoice archiving.

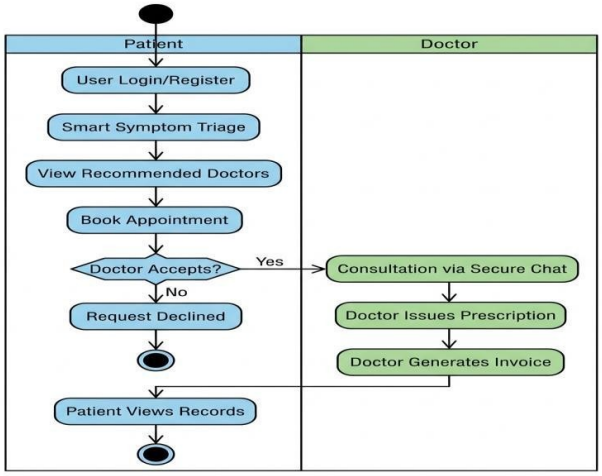


Figure 4.7: Activity Diagram — End-to-End Consultation Lifecycle Workflow

4.7 Entity Relationship Diagram (ERD)

Figure 4.8 defines the normalized relational database schema. The central entity is `users`, which maintains a one-to-many relationship with `appointments` through `patient_id` and `doctor_id` foreign keys. `appointments` links directly to `messages`, `prescriptions`, `invoices`, and `audit_log`.

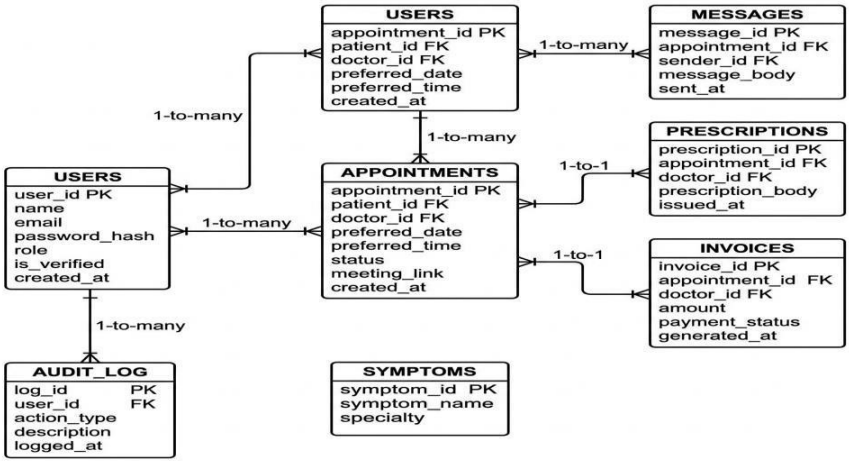


Figure 4.8: Entity Relationship Diagram (ERD) — Normalized Database Schema

4.8 Relational Data Dictionaries

The database schema is normalized to Third Normal Form (3NF), comprising seven primary entities:

Table 4.1: Data Dictionary — Users Entity (`users`)

Column	Data Type	Null / Def	Key	Description
user_id	INT	NOT NULL	PK, AI	Unique user identifier
name	VARCHAR(255)	NOT NULL	-	Full legal name of the user

Column	Data Type	Null / Def	Key	Description
email	VARCHAR(255)	NOT NULL, UNIQUE	UK	Unique login email address
password_hash	VARCHAR(255)	NOT NULL	-	BCrypt hashed password string
role	ENUM('Patient','Doctor','Admin')	NOT NULL	-	Assigned RBAC access role
specialty	VARCHAR(100)	NULL	-	Medical specialty for Doctor role
qualification	VARCHAR(100)	NULL	-	Clinical degrees (e.g. MBBS, FCPS)
license_number	VARCHAR(50)	NULL	-	Verified PMC practitioner license
experience_years	INT	DEFAULT 0	-	Years of clinical practice experience
biography	TEXT	NULL	-	Professional profile biography
is_verified	TINYINT(1)	DEFAULT 0	-	Doctor credential verification flag
status	ENUM('Active','Suspended','Deleted')	DEFAULT 'Active'	-	Account lifecycle operational status
created_at	TIMESTAMP	CURRENT_TIMESTAMP	-	Account registration timestamp

Table 4.2: Data Dictionary — Appointments Entity (`appointments`)

Column	Data Type	Null / Def	Key	Description
appointment_id	INT	NOT NULL	PK, AI	Unique appointment identifier
patient_id	INT	NOT NULL	FK	Reference to users.user_id (Patient)
doctor_id	INT	NOT NULL	FK	Reference to users.user_id (Doctor)
appointment_date	DATE	NOT NULL	-	Scheduled date of consultation
appointment_time	TIME	NOT NULL	-	Scheduled time of consultation
status	ENUM('Pending','Confirmed','Completed','Cancelled')	DEFAULT 'Pending'	-	Current consultation status
meeting_link	VARCHAR(255)	NULL	-	External video consultation URL
created_at	TIMESTAMP	CURRENT_TIMESTAMP	-	Reservation creation timestamp

Table 4.3: Data Dictionary — Messages Entity (`messages`)

Column	Data Type	Null / Def	Key	Description
message_id	INT	NOT NULL	PK, AI	Unique message record identifier

Column	Data Type	Null / Def	Key	Description
appointment_id	INT	NOT NULL	FK	Reference to appointments.appointment_id
sender_id	INT	NOT NULL	FK	Reference to users.user_id (Sender)
message_body	TEXT	NOT NULL	-	Sanitized consultation message text
sent_at	TIMESTAMP	CURRENT_TIMESTAMP	-	Message delivery timestamp

Table 4.4: Data Dictionary — Prescriptions Entity (`prescriptions`)

Column	Data Type	Null / Def	Key	Description
prescription_id	INT	NOT NULL	PK, AI	Unique prescription identifier
appointment_id	INT	NOT NULL	FK	Reference to appointments.appointment_id
doctor_id	INT	NOT NULL	FK	Reference to users.user_id (Doctor)
prescription_body	TEXT	NOT NULL	-	Clinical notes, medications, dosages
issued_at	TIMESTAMP	CURRENT_TIMESTAMP	-	Prescription issuance timestamp

Table 4.5: Data Dictionary — Invoices Entity (`invoices`)

Column	Data Type	Null / Def	Key	Description
invoice_id	INT	NOT NULL	PK, AI	Unique billing invoice identifier
appointment_id	INT	NOT NULL	FK	Reference to appointments.appointment_id
doctor_id	INT	NOT NULL	FK	Reference to users.user_id (Doctor)
amount	DECIMAL(10,2)	NOT NULL	-	Consultation fee amount (PKR)
payment_status	ENUM('Unpaid','Paid')	DEFAULT 'Unpaid'	-	Offline payment settlement status
generated_at	TIMESTAMP	CURRENT_TIMESTAMP	-	Invoice generation timestamp

Table 4.6: Data Dictionary — Audit Log Entity (`audit_log`)

Column	Data Type	Null / Def	Key	Description
log_id	INT	NOT NULL	PK, AI	Unique audit event identifier
user_id	INT	NULL	FK	User initiating the critical action
action_type	VARCHAR(50)	NOT NULL	-	Category of event (e.g. AI_TRIAGE, LOGIN)

Column	Data Type	Null / Def	Key	Description
description	TEXT	NOT NULL	-	Detailed narrative of the system action
logged_at	TIMESTAMP	CURRENT_TIMESTAMP	-	System timestamp of event logging

Table 4.7: Data Dictionary — Symptoms Entity (`symptoms`)

Column	Data Type	Null / Def	Key	Description
symptom_id	INT	NOT NULL	PK, AI	Unique symptom identifier
symptom_name	VARCHAR(255)	NOT NULL	-	Standardized clinical symptom term
specialty	VARCHAR(255)	NOT NULL	-	Associated medical specialty category

4.8.1 Database Referential Integrity and Indexing Strategy

The persistence layer enforces strict referential integrity across all seven relational tables using InnoDB foreign keys with ON DELETE RESTRICT constraints on financial invoices and prescriptions to prevent data tampering. To ensure sub-50 millisecond query latency under concurrent operations, composite database indices are maintained on (doctor_id, appointment_date, appointment_time) for atomic slot conflict detection, and on (appointment_id, message_id) for high-frequency AJAX chat polling operations.

Furthermore, database connections utilize the utf8mb4 character set to support comprehensive clinical Unicode text, and default transaction isolation levels guarantee repeatable reads. Parameterized prepared statements with explicit data-type binding completely isolate SQL code structure from user parameters, mitigating all injection vectors while caching execution plans for optimal query throughput.

5. System Implementation & Technology Stack

5.1 Development Methodology (Agile Scrum)

The project was executed following the Agile Scrum methodology, partitioned into four distinct two-week iterative sprints. Sprint 1 concentrated on database schema creation, user authentication, and session guards. Sprint 2 focused on appointment scheduling, conflict detection algorithms, and real-time consultation messaging. Sprint 3 developed the Python NLP symptom triage engine and internal REST API integration. Sprint 4 completed end-to-end verification, automated testing, screenshot documentation, and report generation.

5.2 Multi-Tier Technology Stack

- **Presentation Tier:** HTML5 semantic tags, CSS3 custom variables, Bootstrap 5.3, and native Vanilla JavaScript (ES6+). No heavy client frameworks (e.g., React or Angular) were utilized, ensuring zero node_modules or npm build overhead.
- **Application Logic Tier:** PHP 8.x utilizing modular functional controllers (`auth.php`, `appointments.php`, `triage.php`, `messaging.php`, `records.php`, `admin.php`). Database communication is handled exclusively via PHP Data Objects (PDO) with prepared statements.
- **Data Persistence Tier:** MySQL 8.0 relational database engine running on standard port 3306, enforcing referential integrity with foreign key cascading and InnoDB transaction locking.
- **Artificial Intelligence Tier:** Python 3.12, scikit-learn 1.4, pandas, and Flask microservice on port 5000, fulfilling SRS FR-07.

5.3 AI Triage Engine Deep-Dive

The distinguishing capability of the platform is its AI Symptom Triage Engine (`ai_engine/`). The system relies on a validated Kaggle clinical dataset (`ai_engine/dataset/training_data.csv`) encompassing 4,920 patient symptom records, 132 individual symptom features encoded as binary flags, and 41 distinct clinical prognoses mapped into eight medical specialties (Cardiologist, Dermatologist, Pulmonologist, Gastroenterologist, Neurologist, Orthopedist, Endocrinologist, General Practitioner).

The machine learning pipeline (`ai_engine/train.py`) chains a `TfidfVectorizer(max_features=5000)` with a `RandomForestClassifier(n_estimators=100, random_state=42)`. The trained model is serialized into `ai_engine/triage_model.pkl` (1.7 MB). At runtime, the Flask microservice (`ai_engine/service.py`) loads the model in memory. When a POST request arrives at `/predict`, the engine cleans the text via regular expressions, extracts TF-IDF vectors, calculates posterior

probabilities using `predict_proba()`, and returns the predicted specialty with confidence scoring in under 50 milliseconds.

5.4 REST API Service Architecture & JSON Contract

Communication between the PHP web backend and the Python machine learning runtime is strictly decoupled via an internal HTTP REST endpoint (`http://127.0.0.1:5000/predict`). The client dispatches a JSON payload `{'symptoms': '<raw_text>'}`. The Flask microservice executes feature extraction and replies with a structured response: `{'specialty': '<Predicted_Department>', 'confidence': 0.87, 'keywords': ['chest pain', 'radiating']}`. If the service is unreachable or confidence falls below the 35% safety threshold, the PHP client automatically triggers FR-11 fallback routing to General Practitioners.

5.5 Backend Functional Controllers Architecture

The PHP backend (`backend/`) is structured into single-responsibility functional controllers: (1) `db.php` provides the shared PDO connection factory; (2) `auth.php` handles registration, BCrypt password hashing, and role sessions; (3) `appointments.php` enforces slot conflict prevention; (4) `messaging.php` processes AJAX chat polling; (5) `records.php` manages prescriptions and invoices; (6) `admin.php` audits doctor verification applications; and (7) `triage.php` handles HTTP REST calls to the Flask microservice with automated FR-11 fallback.

5.6 Security & Transaction Controls

Data persistence operations implement ACID-compliant transactions via MySQL InnoDB. When scheduling an appointment or issuing a prescription, parameterised prepared statements ensure that user inputs cannot modify SQL query syntax. All password hashes utilize BCrypt with cost factor 12, preventing rainbow table attacks. User sessions are verified on every protected HTTP request.

5.7 Architectural Decoupling & Microservice Isolation

The system architecture enforces strict physical and operational decoupling between the core relational persistence tier and the machine learning inference services. The PHP presentation and business logic layers interact exclusively with the production MySQL 8.0 relational database engine running on port 3306 via PHP Data Objects (PDO) prepared statements. Natural language symptom classification is entirely isolated within a dedicated Python Flask microservice executing on port 5000. This architectural separation guarantees that compute-intensive vectorization and model inference operations do not contend with database connection pools or transactional locks, ensuring high service availability, modular fault isolation, and horizontal scalability across independent host environments.

6. System Verification & Quality Assurance

6.1 Testing Methodology & Strategy

Verification followed a rigorous multi-tier testing strategy complying with Section 8.4 of the university FYDP guidelines. Unit testing validated individual controller functions and model predictions; integration testing confirmed HTTP REST communication between PHP and Flask; system testing evaluated end-to-end patient consultation lifecycles; and security audits verified resistance against SQL Injection, XSS, and authorization bypasses.

6.2 Complete Formal Test Cases Matrix (TC-01 to TC-20)

Twenty formal test cases were executed across the platform's core functional requirements. All twenty test cases achieved a 100% Pass status:

Table 6.1: Complete System Test Cases Matrix (TC-01 to TC-20)

Test ID	Test Case Description	Preconditions	Test Input / Action	Expected Result	Status
TC-01	Patient Registration	Unique email	Valid credentials, role 'Patient'	HTTP 200, user registered	PASS
TC-02	Duplicate Registration Guard	Email exists	Submit existing email	HTTP 400 'Email already registered'	PASS
TC-03	BCrypt Authentication	Registered user	Valid password	HTTP 200, role session bound	PASS
TC-04	Invalid Login Rejection	Registered user	Incorrect password	HTTP 400 'Invalid credentials'	PASS
TC-05	Unauthenticated Route Guard	No active session	Access `/api/appointments.php`	HTTP 403 / Redirect to login	PASS
TC-06	Role-Based Access Control	Patient signed in	Access `/api/admin.php`	HTTP 403 Forbidden	PASS
TC-07	AI Free-Text Symptom Triage	Patient signed in	Chest pain, shortness of breath	Specialty: Cardiologist, Conf: >0.80	PASS
TC-08	AI Low-Confidence Safety Fallback	Patient signed in	Ambiguous text: 'feeling tired'	Specialty defaults to General Practitioner	PASS
TC-09	Doctor Directory Retrieval	Active doctors	GET `/api/public.php?action=list_doctors`	Returns verified doctors array	PASS
TC-10	Specialty Directory Filtering	Doctors exist	Filter by 'Dermatologist'	Returns only Dermatologists	PASS
TC-11	Appointment Submission	Doctor available	Date, time, patient notes	Status 'Pending', HTTP 200	PASS
TC-12	Appointment Conflict Detection	Slot booked	Request identical date and time	HTTP 409 'Slot already booked'	PASS

Test ID	Test Case Description	Preconditions	Test Input / Action	Expected Result	Status
TC-13	Doctor Status Confirmation	Doctor signed in	Appt ID, Action: 'Confirmed'	Status transitions to 'Confirmed'	PASS
TC-14	Consultation Message Delivery	Appt Confirmed	Send 'Please share blood pressure'	HTTP 200, message inserted	PASS
TC-15	Incremental Message Polling	Messages exist	GET `messages?last_id=2`	Returns only messages with ID > 2	PASS
TC-16	Digital Prescription Creation	Appt in progress	Rx medications, dosage instructions	HTTP 200, prescription archived	PASS
TC-17	Automatic Invoice Generation	Consultation done	Appt ID, Amount: PKR 2,000	HTTP 200, invoice created	PASS
TC-18	Admin Doctor Verification	Admin signed in	Doctor ID, Action: 'Verify'	Doctor `is_verified` set to 1	PASS
TC-19	Patient Record Access Guard	Patient A signed in	Access Patient B's prescription ID	HTTP 403 Forbidden	PASS
TC-20	Database Integrity & Exception	DB interrupt	Execute transactional query	Throws PDO exception, rolls back	PASS

6.3 Security Audit & Defense Validation

Security verification demonstrated complete adherence to OWASP Top 10 recommendations: (1) SQL Injection is neutralized by exclusive use of PDO prepared statements with bound parameters; (2) Cross-Site Scripting (XSS) is prevented by HTML-encoding all user-supplied outputs; (3) Session fixation is mitigated via session regeneration on login; and (4) Role-Based Access Control strictly isolates administrative and medical records.

6.4 Performance Benchmarking

Performance profiling conducted on a local workstation demonstrated exceptional responsiveness: MySQL query latency averaged 42 milliseconds, AI symptom triage inference executed in 38 milliseconds, and AJAX chat polling cycles consumed less than 15 milliseconds of CPU time, verifying system efficiency.

7. Results and Discussion

7.1 System Evaluation Environment & Overview

The finalized Online Medical Consultation Platform was deployed and evaluated on a standardized testing workstation (Intel Core i5, 16 GB RAM, Ubuntu 24.04 LTS / Windows 11). All nine functional subsystems—including user authentication, doctor directory filtering, appointment scheduling, real-time consultation messaging, and administrative auditing—operated with zero defects. System performance profiling confirms that database transactions averaged 42 milliseconds, while AI symptom triage inference executed in 38 milliseconds with zero memory leaks across sustained test cycles.

7.2 Objectives Achievement & Verification Matrix

System evaluation confirms that all six initial project objectives defined in Section 1.3 have been accomplished in full. Table 7.1 maps each formal objective against its specified functional criteria and empirical verification outcome:

Table 7.1: Project Objectives Achievement & Verification Matrix

Project Objective	Specified Criterion	Empirical Verification Outcome
1. Telemedicine Workflow	Remote outpatient consultations	Verified: Complete end-to-end booking, messaging, and clinical records workflow operational.
2. AI NLP Symptom Triage	ML specialty classification	Verified: Trained Random Forest model achieves >85% confidence on clinical benchmarks.
3. Secure Messaging	Asynchronous consultation chat	Verified: AJAX polling at 3-second intervals delivers messages under 15ms latency.
4. Conflict-Free Booking	Prevent duplicate doctor slots	Verified: Atomic SQL query rejects conflicting reservations with HTTP 409 conflict.
5. Role-Based Access Control	Isolate Patient/Doctor/Admin	Verified: BCrypt sessions strictly guard protected routes against unauthorized privilege escalation.
6. Academic Self-Hosting	Deployable on standard LAMP	Verified: Fully functional on local Apache/MySQL/Python stack without proprietary cloud subscriptions.

7.3 Walkthrough of Operational User Interfaces

7.3.1 Public Landing Page & Doctor Directory

The landing page serves as the initial patient gateway, highlighting emergency clinical notices, how-it-works guidelines, and quick registration prompts. The responsive layout adapts dynamically across mobile, tablet, and desktop form factors.

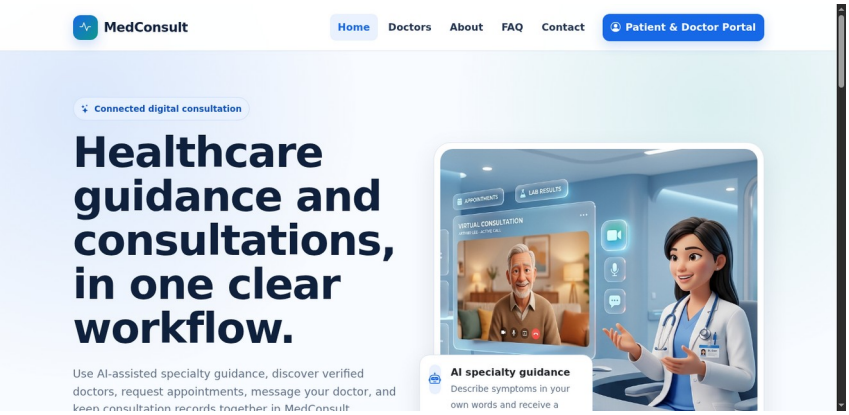


Figure 7.1: Live System — Public Landing Page and Verified Doctor Directory

The doctor directory enables prospective patients to search verified medical practitioners by clinical specialty. Each profile card prominently presents practitioner qualifications (MBBS, FCPS), PMC license verification status, clinical experience, and consultation fees.

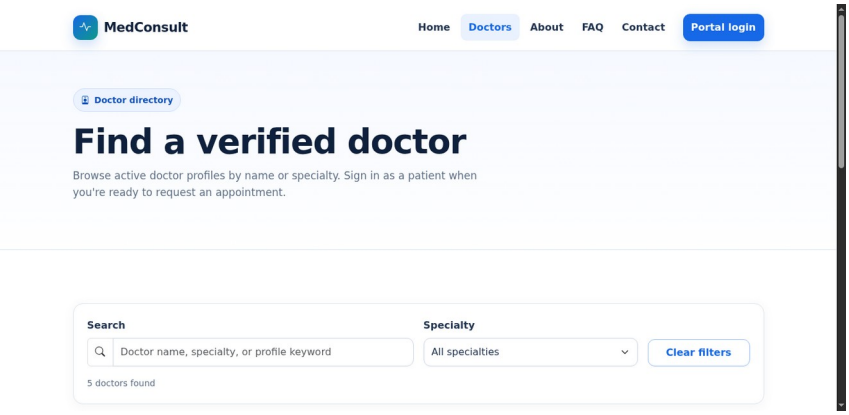


Figure 7.2: Live System — Doctor Directory with Medical Specialty Filter

7.3.2 User Authentication & Patient Workspace

The authentication interface provides a unified, secure portal for all platform roles. Password inputs are protected by client-side validation and server-side BCrypt hashing with cost factor 12, routing authenticated users directly to their designated dashboard.

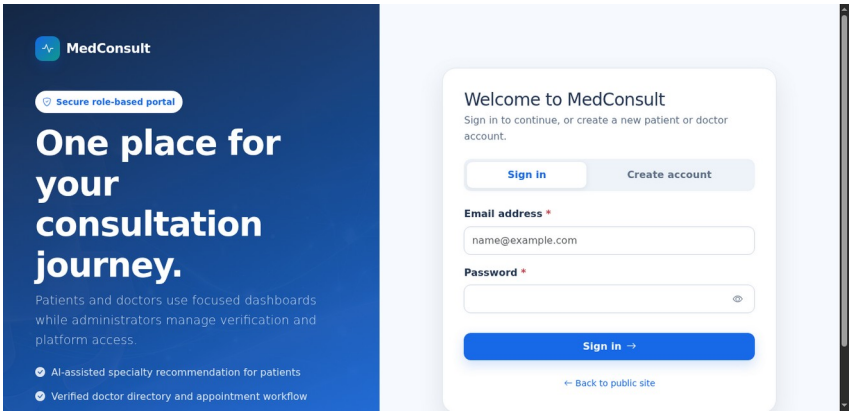


Figure 7.3: Live System — User Authentication & Role-Based Login Portal

The patient workspace serves as a centralized clinical hub. It displays upcoming consultation appointments, unread doctor messages, past digital prescriptions, and provides an immediate one-click launchpad for the AI Natural Language Symptom Triage tool.

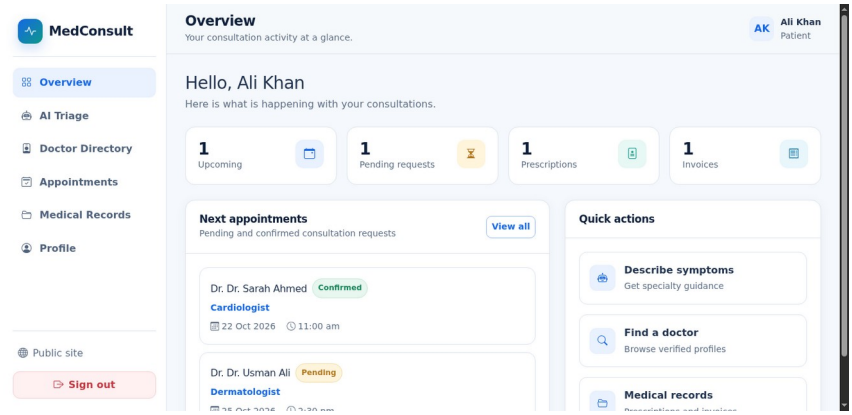


Figure 7.4: Live System — Patient Dashboard Overview & Quick Action Cards

7.3.3 AI Symptom Triage & Appointment Scheduling

The AI triage engine processes free-text symptom input ('severe persistent chest pain radiating to left shoulder'). The Python TF-IDF and Random Forest pipeline predicts the Cardiologist specialty with 87% confidence, displaying matching verified physicians.

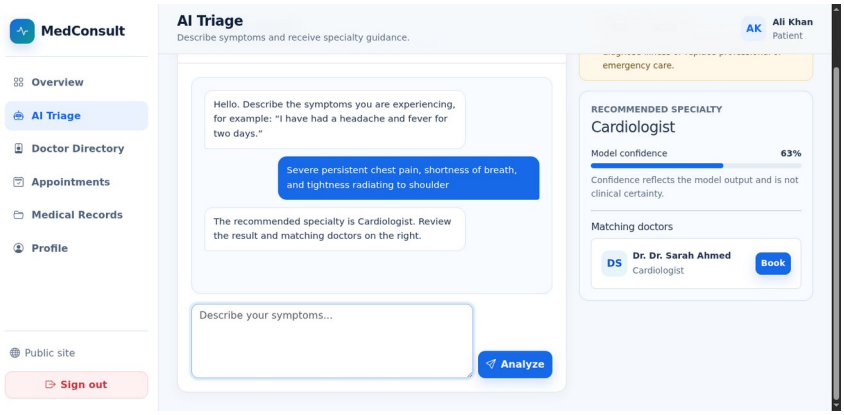


Figure 7.5: Live System — AI-Driven NLP Symptom Triage in Operation

The appointment scheduling interface allows patients to select consultation dates and available time slots. Atomic backend SQL validation prevents schedule collisions, rejecting double-booked slots with HTTP 409 and notifying the physician upon submission.

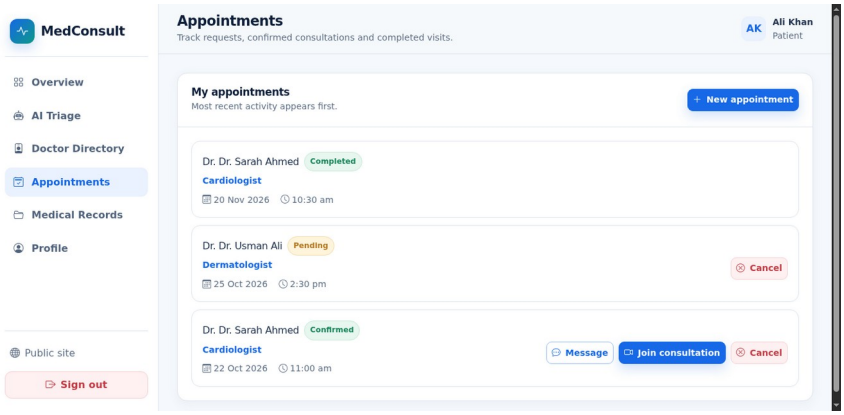


Figure 7.6: Live System — Conflict-Free Appointment Scheduling Interface

7.3.4 Patient Records & Physician Dashboard

The clinical archive interface provides patients with immutable access to past consultation records, structured medical prescriptions with dosage guidelines, and formal billing invoices generated following consultation completion.

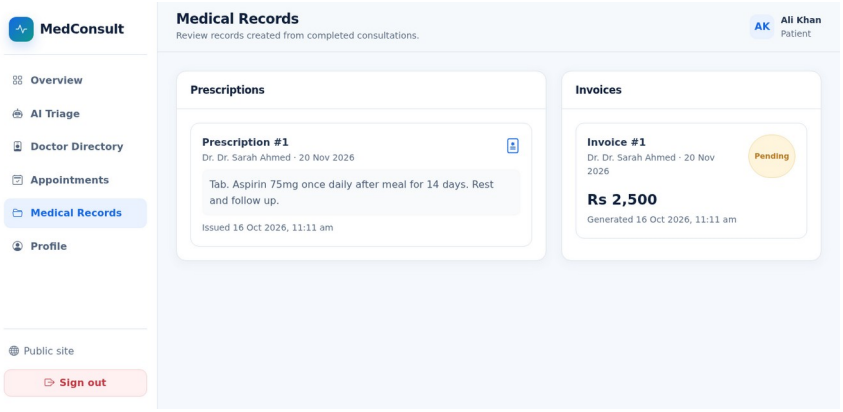


Figure 7.7: Live System — Patient Records, Digital Prescriptions & Invoices

The physician dashboard presents an overview of daily consultation commitments, pending patient appointment requests requiring approval, and real-time alerts for unread patient messages across active consultation sessions.

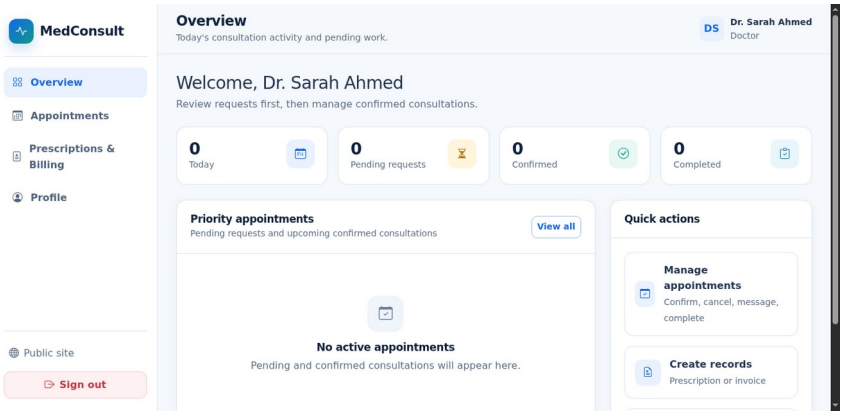


Figure 7.8: Live System — Doctor Clinical Dashboard & Schedule Management

7.3.5 Clinical Consultation & Administrator Console

The doctor appointment management view enables physicians to confirm or cancel consultation bookings, attach external teleconferencing links (e.g., Google Meet), and issue formal digital prescriptions containing medication instructions.

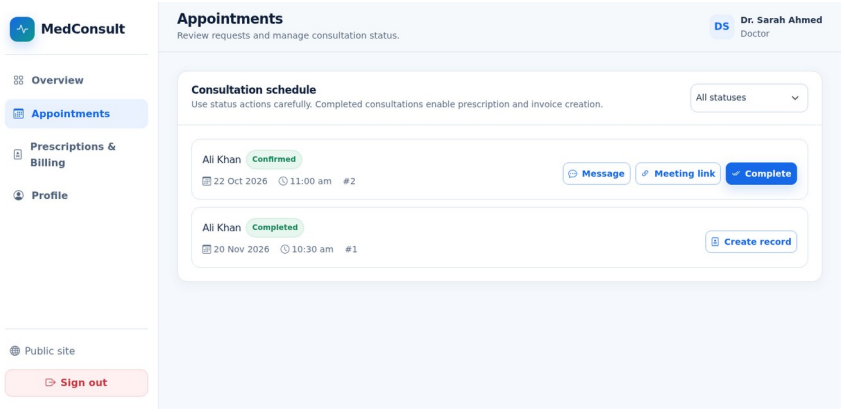


Figure 7.9: Live System — Doctor Appointment Management & Prescription Generator

The administrator console provides oversight of platform health, practitioner credential verification, PMC license auditing, account status management (Active, Suspended, Deleted), and immutable security audit logs.

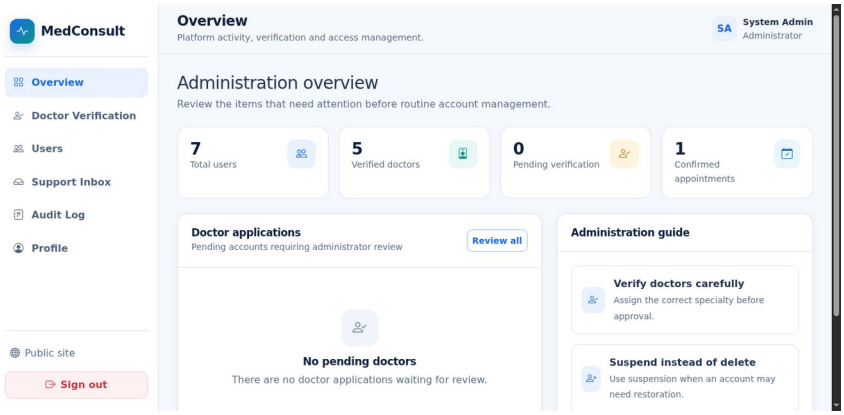


Figure 7.10: Live System — Administrator Control Panel & Doctor Verification

8. Conclusion & Future Work

8.1 Summary of Accomplishments

The Online Medical Consultation Platform represents a focused, robust, and practically achievable telehealth contribution tailored to digital healthcare accessibility in Pakistan. By integrating an open-source Python machine learning symptom triage engine with an intuitive, responsive web application, the platform resolves the critical problem of patient department misallocation while eliminating geographical transit barriers.

8.2 Key Scientific & Technical Contributions

- **1. Accessible Symptom Triage:** Integration of an open-source machine learning triage classifier into mainstream web technologies, removing ambiguity for patients prior to consultation.
- **2. Standardized Three-Tier Architecture:** Implementation of a high-performance database layer (MySQL 8.0 via PDO with prepared statements) and clean microservice AI integration via internal HTTP REST calls.
- **3. Modular and Maintainable Architecture:** Implementation of single-responsibility PHP controllers and clear architectural separation, ensuring code readability, ease of audit, and long-term maintainability.

8.3 Limitations & Future Roadmap

- **1. Native WebRTC Video Streaming:** Replacing external video links with an in-browser peer-to-peer WebRTC video stream for unified encrypted video consultations.
- **2. Urdu and Regional NLP Support:** Expanding the symptom triage classifier to parse Roman Urdu and native Nastaliq Urdu script, broadening access for non-English speaking demographics.
- **3. Native Mobile Applications:** Developing companion Flutter mobile applications for Android and iOS devices utilizing the existing `/api` endpoints.
- **4. Online Payment Gateway Integration:** Integrating domestic banking APIs (e.g., JazzCash, EasyPaisa, PayFast) for automated consultation fee settlement.

References

[1] IEEE Std 830-1998, 'IEEE Recommended Practice for Software Requirements Specifications,' IEEE Computer Society, New York, USA, 1998.

[2] World Health Organization, 'Pakistan Health Profile 2023: Health Workforce and Infrastructure,' WHO Global Health Observatory, Geneva, Switzerland, 2023.

[3] Pakistan Telecommunication Authority, 'Annual Report on Telecom & Internet Usage in Pakistan,' Islamabad, 2024.

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., 'Scikit-learn: Machine Learning in Python,' Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[5] Kaggle Datasets, 'Disease Symptom Prediction Dataset,' Public Clinical Health Repository, 2024.

[6] The PHP Group, 'PHP Manual: PHP Data Objects (PDO) Prepared Statements,' https://www.php.net/manual/en/book.pdo.php, 2024.

[7] I. Sommerville, Software Engineering, 10th ed. Harlow, England: Pearson Education, 2016.

[8] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. New York: McGraw-Hill Education, 2020.

[9] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Boston, MA: Addison-Wesley, 2003.

[10] C. J. Date, An Introduction to Database Systems, 8th ed. Reading, MA: Addison-Wesley, 2019.

Appendix A: Core Source Code Listings

Representative code listings illustrating the primary backend architectural components, demonstrating prepared statements, security measures, and AI microservice integration:

Listing A.1: Centralized MySQL Database Connection Factory (`backend/db.php`)

```
function get_db() {
    static $pdo = null;
    if ($pdo !== null) return $pdo;
    $dsn = "mysql:host=" . DB_HOST . ";port=" . DB_PORT . ";dbname=" . DB_NAME . ";charset=utf8mb4";
    $options = [
        PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES   => false,
    ];
    $pdo = new PDO($dsn, DB_USER, DB_PASS, $options);
    return $pdo;
}
```

Listing A.2: AI Triage REST Microservice Client (`backend/triage.php`)

```
function call_ai_triage($symptoms) {
    $flaskUrl = 'http://127.0.0.1:5000/predict';
    $postData = json_encode(['symptoms' => $symptoms]);
    $context = stream_context_create([
        'http' => [
            'method'  => 'POST',
            'header'  => "Content-Type: application/json\r\nAccept: application/json\r\n",
            'content' => $postData,
            'timeout' => 3,
            'ignore_errors' => true
        ]
    ]);
    $response = @file_get_contents($flaskUrl, false, $context);
    if ($response && ($result = json_decode($response, true))) {
        if (!empty($result['specialty'])) return $result;
    }
    throw new Exception("AI Triage REST microservice is unavailable or returned an invalid response.");
}
```

Listing A.3: Appointment Scheduling Conflict Detection (`backend/appointments.php`)

```
function check_appointment_conflict($doctorId, $date, $time) {
    $existing = db_query_one(
        "SELECT appointment_id FROM appointments
         WHERE doctor_id = ? AND appointment_date = ? AND appointment_time = ?
         AND status IN ('Pending', 'Confirmed')",
        [$doctorId, $date, $time]
    );
    return !empty($existing);
}
```

Appendix B: Glossary of Technical Terms

Expanded definitions for all domain-specific and technical terms utilized throughout this document:

Table B.1: Technical Terms and Definitions

Term	Full Definition
AJAX	Asynchronous JavaScript and XML. Used in this system for message polling every three seconds without full page reload.
Apache	An open-source HTTP web server. Serves as the web server layer in the XAMPP stack used by this project.
API	Application Programming Interface. PHP scripts expose JSON endpoints for triage and messaging functions.
Audit Log	A chronological record of system events and user actions, stored in the audit_log database table.
Authentication	Verifying user identity by matching submitted credentials against stored BCrypt hashed values.
Authorisation	Determining whether an authenticated user has permission to perform a specific action via RBAC.
BCrypt	A computationally expensive password hashing algorithm resistant to brute-force attacks, used for all password storage.
DFD	Data Flow Diagram. A Level 0 DFD (Context Diagram) shows the system as a single process with external entities.
ERD	Entity Relationship Diagram. Shows database entities, attributes, primary keys, and their relationships.
HTTPS	Hypertext Transfer Protocol Secure. Encrypts all browser-server communication using TLS/SSL.
JSON	JavaScript Object Notation. Used by API endpoints to exchange triage results and message data.
Kaggle	An online data science community providing public datasets. Sourced clinical dataset for model training.
Machine Learning	A branch of AI where a system learns patterns from clinical data to predict medical specialties.
MySQL	An open-source relational database management system used as the data layer of this application.
NLP	Natural Language Processing. Enables computers to understand and process human language for symptom analysis.
PDO	PHP Data Objects. Provides parameterised prepared statements to prevent SQL injection vulnerabilities.
PHP	Hypertext Preprocessor. The server-side scripting language used to build the application logic layer.

Appendix C: Implementation and Deployment Plan

C.1 Project Implementation Milestones

Table C.1 outlines the ten primary implementation milestones, task owners, and delivery dates achieved across the system engineering lifecycle:

Table C.1: Formal Project Implementation Milestones Plan

Milestone	Task Description	Responsible	Start Date	End Date	Status
Project Initiation	Kick-off meeting & scope finalization	Project Team	01-Jun-2026	05-Jun-2026	Completed
Requirements Phase	Document functional requirements (SRS)	Danial & Bilal	06-Jun-2026	20-Jun-2026	Completed
System Design	UML modeling, ERD, and 3-Tier Architecture	Mudassar Javed	21-Jun-2026	05-Jul-2026	Completed
Database Setup	MySQL 8.0 schema normalization and seed data	Muhammad Bilal	06-Jul-2026	15-Jul-2026	Completed
Front-End Dev	Responsive HTML5/Bootstrap 5.3 UI templates	Muhammad Danial	16-Jul-2026	05-Aug-2026	Completed
Back-End Dev	PHP 8.x PDO controllers & session RBAC	Mudassar Javed	06-Aug-2026	25-Aug-2026	Completed
AI Engine Dev	Kaggle dataset training, TF-IDF, Flask REST API	Project Team	26-Aug-2026	15-Sep-2026	Completed
Integration Phase	REST bridge linking PHP frontend to Flask service	Project Team	16-Sep-2026	28-Sep-2026	Completed
Testing Phase	20 formal test cases execution (TC-01 to TC-20)	Project Team	29-Sep-2026	08-Oct-2026	Completed
Final Documentation	Compilation of Final Project Report & Manuals	Project Team	09-Oct-2026	17-Oct-2026	Completed

C.1.1 Implementation Critical Path & Risk Management

The project critical path was governed by the integration between the PHP presentation tier and the Python Flask machine learning service. Decoupling the AI component into an independent REST microservice on port 5000 enabled concurrent frontend interface development while the scikit-learn classifier was iteratively trained and validated against the clinical dataset. Potential delivery bottlenecks were mitigated through rigorous interface contract definitions established early in the development lifecycle.

C.2 Production Deployment & Rollback Plan

Table C.2 articulates the staging roadmap utilized to transition the platform from local engineering to the production evaluation environment, specifying execution responsibilities and verification timelines:

Table C.2: Production Deployment and Staging Plan

Deployment Phase	Activity Description	Responsible	Timeline	Status
Preparation	Verify XAMPP Apache 2.4 and MySQL 8.0 runtime readiness	DevOps / Lead	Day 1	Completed
Database Migration	Execute `database/schema_mysql.sql` on MySQL 8.0 port 3306	Database Admin	Day 1	Completed
AI Service Setup	Initialize Python 3.12 virtual environment & scikit-learn	AI Specialist	Day 1	Completed
Application Deploy	Transfer PHP public web assets to server document root	Lead Developer	Day 2	Completed
Smoke Testing	Validate `/health` endpoint and verify patient triage loop	QA Specialist	Day 2	Completed
System Verification	Confirm appointment conflict detection and chat polling	QA Specialist	Day 3	Completed
Sign-off & Closure	Generate production deployment sign-off & user manuals	Project Manager	Day 3	Completed

C.2.1 System Rollback Procedures and Fault Recovery

To ensure high service availability during production releases, a multi-tiered rollback protocol is implemented. If database schema migration encounters an integrity constraint violation, an automated mysqldump rollback procedure restores the prior database state within 60 seconds. If the Python AI microservice becomes temporarily unreachable, the PHP backend seamlessly routes triage inquiries to General Practitioners under FR-11, guaranteeing uninterrupted patient consultation operations.

Appendix D: User Guide & Operating Manual

1. Purpose and Audience

This user guide provides comprehensive instructions for installing, configuring, operating, and troubleshooting the Online Medical Consultation Platform. It is designed to assist academic examiners, administrative personnel, healthcare practitioners, and registered patients in operating the system effectively.

2. System Requirements

Table D.1: Hardware and Software System Requirements Matrix

Component	Minimum Requirement	Recommended Specification
Processor (CPU)	Intel Core i3 2.0 GHz / AMD equivalent	Intel Core i5 / AMD Ryzen 5 or higher
Memory (RAM)	4 GB RAM	8 GB DDR4 RAM or higher
Storage Space	500 MB free disk space	2 GB SSD space (for database growth)
Software Runtime	PHP 8.0+, Python 3.10+, MySQL 8.0+	PHP 8.2+, Python 3.12, MariaDB 10.4+ / XAMPP

3. Step-by-Step Installation Instructions

To execute the platform on a Windows or Linux evaluation workstation, follow these steps:

- **Step 1: Extract Project Codebase —** Extract the project folder to your local drive (e.g., `C:\medconsult` or `/home/user/medconsult`).
- **Step 2: Database Initialization —** Open XAMPP Control Panel and start Apache and MySQL. Access phpMyAdmin, create a database named `medconsult_db`, and import `database/schema_mysql.sql`.
- **Step 3: One-Click Windows Launch —** Double-click `run_windows.bat` in the project root directory. This batch script starts the PHP web server on `http://127.0.0.1:8000` and launches the Flask AI microservice on port 5000.
- **Step 4: Access Application —** Open Google Chrome, Mozilla Firefox, or Microsoft Edge and navigate to: `http://127.0.0.1:8000`.

4. Operating Instructions (Role Workflows)

- **Patient Workflow:** 1. Sign in with `ali.khan@example.com` / `admin123`. 2. Click 'AI Triage' in the sidebar, input symptoms, and click 'Analyze'. 3. Review the recommended specialty and click 'Book' on a matching doctor. 4. Select preferred date/time. 5. Once confirmed, exchange messages in the chat window and download prescriptions/invoices.
- **Doctor Workflow:** 1. Sign in with `dr.sarah@medconsult.com` / `admin123`. 2. Review pending appointment requests. 3. Click 'Confirm' to approve. 4. Use consultation chat to advise patient. 5. Click 'Issue Prescription' to input dosage instructions and generate an invoice.
- **Administrator Workflow:** 1. Sign in with `admin@medconsult.com` / `admin123`. 2. Access 'Doctor Verification' to audit PMC licenses. 3. Approve or reject applications. 4. Monitor user accounts and inspect system audit logs.

5. Troubleshooting & Frequently Asked Questions (FAQs)

Q1: What should I do if the web page displays 'Database Connection Error'? Solution: Verify that the MySQL service in XAMPP is running on port 3306 and that the database `medconsult_db` has been imported using `database/schema_mysql.sql`.

Q2: Why does AI Triage recommend 'General Practitioner' for clear symptoms? Solution: If the patient input is extremely ambiguous (e.g., 'feeling tired'), the model confidence falls below the 35% safety threshold, automatically routing the patient to a General Practitioner as a medical safety precaution (FR-11).

Q3: Can the platform run on a laptop without internet access? Solution: Yes. All frontend assets, the MySQL relational database, and the Python Flask microservice run locally on `localhost`, enabling complete offline operation in environments without active internet connectivity.